

Credit Card Fraud Detection Using Anomaly Detection Techniques

An end-to-end machine learning project that detects fraudulent credit card transactions, compares supervised and unsupervised models, and serves predictions through a web API with an interactive **Transaction Risk Dashboard**.

Table of Contents

1. [Overview](#)
2. [Key Features](#)
3. [Tech Stack](#)
4. [System Architecture](#)
5. [Project Structure](#)
6. [Dataset](#)
7. [Installation & Setup](#)
8. [Machine Learning Pipeline](#)
9. [Model Comparison](#)
10. [Evaluation Metrics](#)
11. [Database \(SQL\) Design](#)
12. [API Reference](#)
13. [Dashboard](#)
14. [Results](#)
15. [Future Improvements](#)

Overview

Credit card fraud is rare but costly. In typical datasets, fewer than **0.2%** of transactions are fraudulent, which makes this a highly **imbalanced classification / anomaly detection** problem where plain accuracy is misleading.

This project:

- Trains and compares **four models**: Logistic Regression, Random Forest, XGBoost, and Isolation Forest.
- Handles class imbalance using class weights, SMOTE, and threshold tuning.
- Evaluates with metrics that matter for fraud: **Precision, Recall, F1-score, ROC-AUC, PR-AUC, and Confusion Matrix**.
- Outputs a **fraud probability** and **risk level** for every transaction.
- Stores transactions and predictions in a **SQL database**.
- Exposes a **Flask/FastAPI** backend and a **HTML/CSS/JavaScript** dashboard with **Chart.js/Plotly** visualizations.

Key Features

Feature	Description
Multi-model comparison	Logistic Regression vs Random Forest vs XGBoost vs Isolation Forest
Imbalance handling	<code>class_weight</code> , <code>scale_pos_weight</code> , SMOTE, undersampling
Rich evaluation	Precision, Recall, F1, ROC-AUC, PR-AUC, Confusion Matrix
Fraud probability	Per-transaction probability score (0–1)
Risk classification	Low / Medium / High / Critical risk bands
REST API	Single and batch prediction endpoints
SQL storage	Transaction history, predictions, and model runs
Interactive dashboard	Live charts, filters, model comparison, risk table
Explainability	Feature importance (and optional SHAP values)
Reproducibility	Fixed seeds, saved models, <code>requirements.txt</code>

Tech Stack

Language & Data

- Python 3.9+
- Pandas, NumPy

Machine Learning

- Scikit-learn (Logistic Regression, Random Forest, Isolation Forest, metrics, preprocessing)
- XGBoost
- imbalanced-learn (SMOTE)
- Joblib (model persistence)

Backend

- Flask **or** FastAPI (choose one)
- SQLAlchemy + SQLite / PostgreSQL / MySQL

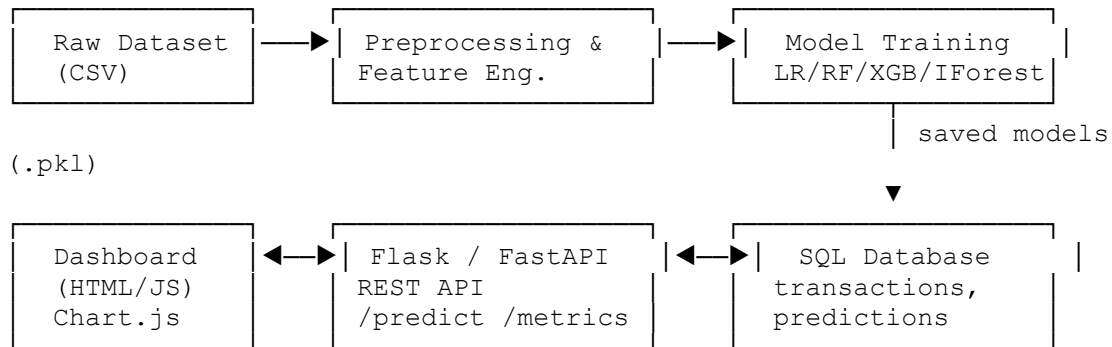
Frontend

- HTML5, CSS3, JavaScript
- Chart.js and/or Plotly.js

Database

- SQL (SQLite for development, PostgreSQL/MySQL for production)

System Architecture



Dataset

Source: [Credit Card Fraud Detection – Kaggle \(ULB Machine Learning Group\)](#)

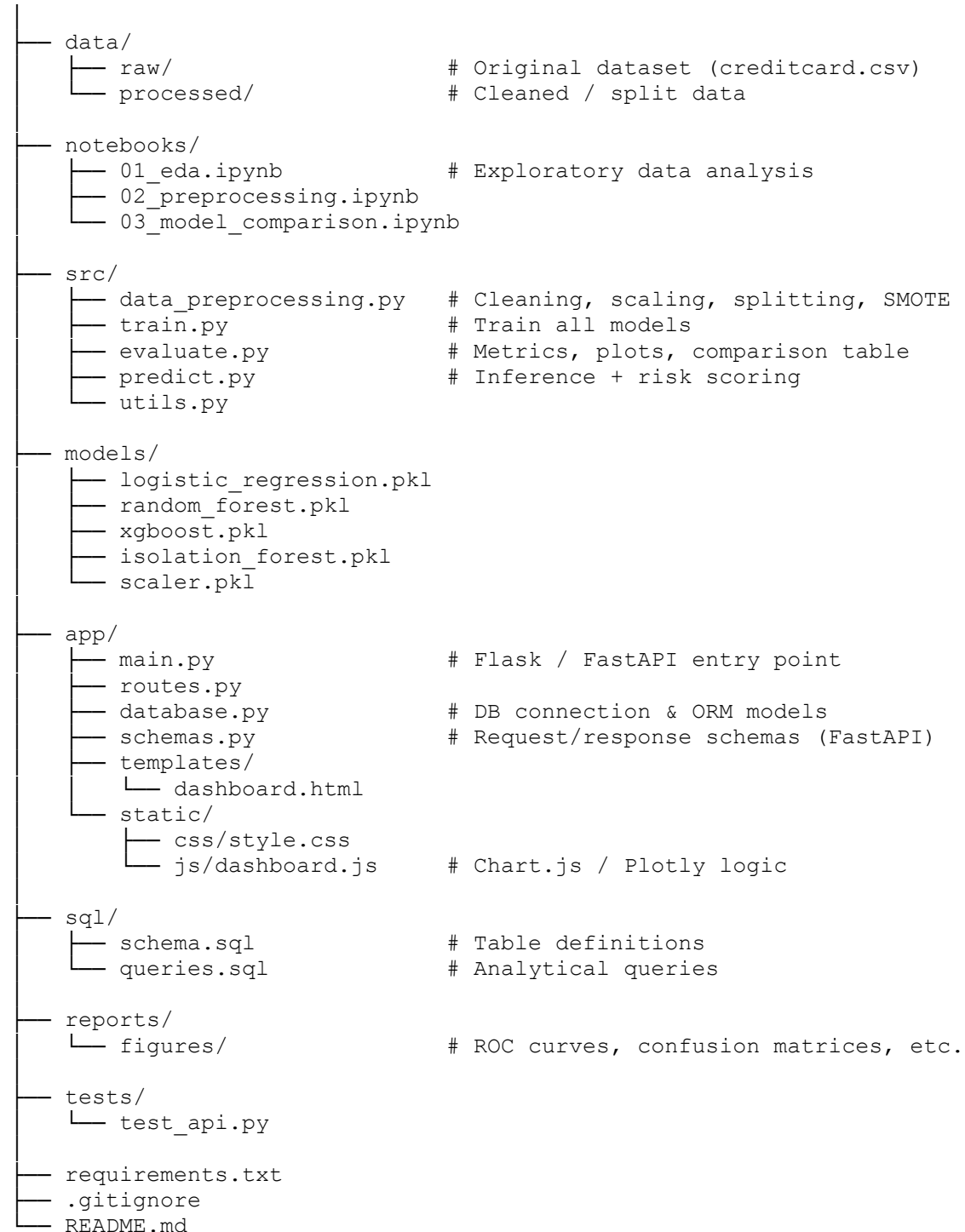
Property	Value
Transactions	284,807
Fraudulent	492 ($\approx 0.172\%$)
Period	2 days (European cardholders, September 2013)
Features	Time, V1–V28 (PCA-transformed), Amount
Target	Class (0 = legitimate, 1 = fraud)

Features V1–V28 are anonymized PCA components. Only Time and Amount are in their original form

Download `creditcard.csv` and place it in `data/raw/`.

Project Structure

credit-card-fraud-detection/



Installation & Setup

1. Clone the repository

```
git clone https://github.com/<your-username>/credit-card-fraud-detection.git
cd credit-card-fraud-detection
```

2. Create a virtual environment

```
python -m venv venv

# Linux / macOS
source venv/bin/activate

# Windows
venv\Scripts\activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

requirements.txt

```
pandas
numpy
scikit-learn
xgboost
imbalanced-learn
joblib
matplotlib
seaborn
plotly
shap
flask          # or: fastapi + uvicorn
sqlalchemy
python-dotenv
pytest
```

4. Add the dataset

```
mkdir -p data/raw
# Place creditcard.csv inside data/raw/
```

5. Initialize the database

```
sqlite3 fraud.db < sql/schema.sql
```

6. Train the models

```
python src/train.py
```

7. Evaluate and compare models

```
python src/evaluate.py
```

8. Run the application

Flask

```
python app/main.py
```

FastAPI

```
uvicorn app.main:app --reload
```

Open **http://localhost:5000** (Flask) or **http://localhost:8000** (FastAPI) in your browser.

Machine Learning Pipeline

1. Exploratory Data Analysis

- Class distribution and imbalance ratio
- Distribution of `Amount` and `Time` for fraud vs. legitimate
- Correlation heatmap of features
- Outlier analysis

2. Preprocessing

- Scale `Amount` and `Time` (`StandardScaler` / `RobustScaler`)
- Remove duplicates, check for missing values
- **Stratified** train/test split (e.g., 80/20) so fraud appears in both sets

3. Handling Class Imbalance

Technique	Used With
<code>class_weight='balanced'</code>	Logistic Regression, Random Forest
<code>scale_pos_weight</code>	XGBoost
SMOTE (training set only)	Optional for all supervised models
Random undersampling	Optional baseline
Threshold tuning	All models

Apply SMOTE **only to the training set** after the split, to avoid data leakage.

4. Model Training

- Hyperparameter tuning with `GridSearchCV` / `RandomizedSearchCV`
- Stratified K-Fold cross-validation
- Scoring on **PR-AUC** / **F1**, not accuracy

5. Risk Scoring

Each transaction receives a fraud probability, mapped to a risk level:

Fraud Probability Risk Level

0.00 – 0.30	☐ Low
0.30 – 0.60	☐ Medium
0.60 – 0.85	☐ High
0.85 – 1.00	☐ Critical

(Thresholds are configurable in `src/predict.py`.)

Model Comparison

Model	Type	Strengths	Limitations
Logistic Regression	Supervised (linear)	Fast, interpretable baseline	Struggles with non-linear patterns
Random Forest	Supervised (ensemble)	Robust, handles non-linearity, feature importance	Slower, larger memory footprint
XGBoost	Supervised (boosting)	Typically best performance on tabular data	More hyperparameters to tune
Isolation Forest	Unsupervised (anomaly detection)	Needs no labels, detects novel fraud patterns	Lower precision, sensitive to contamination

Isolation Forest note: it outputs an anomaly score rather than a probability. Scores are normalized (e.g., min–max) to a 0–1 range so they can be compared and displayed alongside the supervised models. Set `contamination` close to the real fraud rate (≈ 0.0017).

Evaluation Metrics

Accuracy is **not** used as the main metric, since a model predicting "not fraud" every time would score ~99.8%.

Metric	Why It Matters
Precision	Of the transactions flagged as fraud, how many really are? (Avoids false alarms)
Recall	Of all real frauds, how many were caught? (Avoids missed fraud)
F1-Score	Harmonic mean of Precision and Recall
ROC-AUC	Ability to separate fraud from legitimate across all thresholds
PR-AUC	More informative than ROC-AUC on highly imbalanced data
Confusion Matrix	Shows TP, FP, TN, FN counts directly
Fraud Probability	Per-transaction risk score for ranking and alerting

```
from sklearn.metrics import (  
    precision_score, recall_score, f1_score,  
    roc_auc_score, average_precision_score, confusion_matrix  
)
```

```
y_pred = (y_proba >= threshold).astype(int)
```

```
precision = precision_score(y_test, y_pred)  
recall    = recall_score(y_test, y_pred)  
f1        = f1_score(y_test, y_pred)  
roc_auc   = roc_auc_score(y_test, y_proba)  
pr_auc    = average_precision_score(y_test, y_proba)  
cm        = confusion_matrix(y_test, y_pred)
```

Generated visuals (saved to reports/figures/):

- ROC curves (all models on one plot)
- Precision–Recall curves
- Confusion matrix heatmaps
- Feature importance charts
- Metrics comparison bar chart

Database (SQL) Design

sql/schema.sql

```
CREATE TABLE IF NOT EXISTS transactions (
    transaction_id INTEGER PRIMARY KEY AUTOINCREMENT,
    time_elapsed REAL,
    amount REAL NOT NULL,
    v1 REAL, v2 REAL, v3 REAL, v4 REAL, v5 REAL, v6 REAL, v7 REAL,
    v8 REAL, v9 REAL, v10 REAL, v11 REAL, v12 REAL, v13 REAL, v14 REAL,
    v15 REAL, v16 REAL, v17 REAL, v18 REAL, v19 REAL, v20 REAL, v21 REAL,
    v22 REAL, v23 REAL, v24 REAL, v25 REAL, v26 REAL, v27 REAL, v28 REAL,
    actual_class INTEGER, -- ground truth, if known
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS predictions (
    prediction_id INTEGER PRIMARY KEY AUTOINCREMENT,
    transaction_id INTEGER NOT NULL,
    model_name TEXT NOT NULL,
    fraud_probability REAL NOT NULL,
    predicted_class INTEGER NOT NULL,
    risk_level TEXT CHECK (risk_level IN
('Low', 'Medium', 'High', 'Critical')),
    predicted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (transaction_id) REFERENCES transactions(transaction_id)
);

CREATE TABLE IF NOT EXISTS model_metrics (
    run_id INTEGER PRIMARY KEY AUTOINCREMENT,
    model_name TEXT NOT NULL,
    precision REAL,
    recall REAL,
    f1_score REAL,
    roc_auc REAL,
    pr_auc REAL,
    trained_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Example analytical queries (sql/queries.sql)

```
-- Top 10 highest-risk transactions
SELECT t.transaction_id, t.amount, p.model_name, p.fraud_probability, p.risk_level
FROM predictions p
JOIN transactions t ON t.transaction_id = p.transaction_id
ORDER BY p.fraud_probability DESC
LIMIT 10;

-- Count of transactions per risk level
SELECT risk_level, COUNT(*) AS total
FROM predictions
GROUP BY risk_level;

-- Average fraud amount vs. legitimate amount
SELECT actual_class, ROUND(AVG(amount), 2) AS avg_amount
FROM transactions
GROUP BY actual_class;

-- Latest metrics per model
SELECT * FROM model_metrics ORDER BY trained_at DESC;
```

API Reference

Base URL: `http://localhost:5000` (Flask) / `http://localhost:8000` (FastAPI)

Method	Endpoint	Description
GET	/	Serves the dashboard
POST	/api/predict	Predict fraud for a single transaction
POST	/api/predict/batch	Predict fraud for multiple transactions (JSON/CSV)
GET	/api/metrics	Return evaluation metrics for all models
GET	/api/transactions	List stored transactions with risk scores
GET	/api/stats	Summary stats (totals, fraud rate, risk distribution)
GET	/api/health	Health check

Example: Single Prediction

Request

```
curl -X POST http://localhost:8000/api/predict \
-H "Content-Type: application/json" \
-d '{
  "model": "xgboost",
  "features": {
    "Time": 406, "Amount": 149.62,
    "V1": -1.36, "V2": -0.07, "...": "...", "V28": -0.02
  }
}'
```

Response

```
{
  "model": "xgboost",
  "fraud_probability": 0.9231,
  "prediction": "Fraud",
  "risk_level": "Critical"
}
```

FastAPI users get auto-generated interactive docs at `/docs`.

Dashboard

The web dashboard (HTML/CSS/JavaScript + Chart.js/Plotly) provides:

- **KPI cards:** total transactions, flagged frauds, fraud rate, total amount at risk
- **Model comparison chart:** grouped bar chart of Precision, Recall, F1, ROC-AUC
- **ROC & Precision–Recall curves:** interactive, all models overlaid
- **Confusion matrix:** heatmap with model selector dropdown
- **Risk distribution:** donut/pie chart of Low / Medium / High / Critical
- **Transaction risk table:** sortable and filterable, with fraud probability and color-coded risk badges
- **Live prediction form:** enter or upload a transaction and get an instant risk score
- **Time series view:** fraud activity over time
- **Feature importance:** top contributing features per model

The screenshot shows a web browser window with the title 'Fraud Detection Dashboard' and the URL 'localhost:8501'. The dashboard has a dark theme. On the left, there is a sidebar with the following sections:

- Dashboard**
- Choose input**
 - ☒ Single transaction
 - ☐ Batch CSV
- Model**
 - Random Forest (selected)
 - Random Forest
 - Logistic Regression
 - Isolation Forest

The main content area has a header with a shield icon and the text 'Credit Card Fraud Detection' and 'Transaction risk assessment using your trained machine-learning models'. Below this is a section titled 'Enter transaction details' with a subtitle 'Enter Time, V1-V28 and Amount. For exact parity with training, use the same preprocessing as your notebook.' The form contains input fields for 'Time' (0.00), 'Amount' (100.00), and 'Anonymized PCA features (V1-V28)' (V1: 0.000000, V2: 0.000000, V3: 0.000000, V4: 0.000000, V5: 0.000000, V6: 0.000000, V7: 0.000000, V8: 0.000000). The Windows taskbar at the bottom shows the time as 11:48 and the temperature as 31°C.

The screenshot shows the same web browser window as the previous one, but with the 'Batch CSV' option selected in the 'Choose input' section. The main content area now displays a section titled 'Batch transaction analysis' with a subtitle 'Upload CSV containing Time, V1-V28, Amount'. There is an 'Upload' button with a file icon and the text '200MB per file - CSV'. Below this, there is a disclaimer: 'Models: Logistic Regression, Random Forest, XGBoost and Isolation Forest | For educational/demo use; validate thresholds and preprocessing before operational use.' The Windows taskbar at the bottom shows the time as 11:48 and the temperature as 31°C.

Results

Fill this table in after running `python src/evaluate.py`. Values depend on the random seed, split, and tuning.

Model	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	–	–	–	–	–
Random Forest	–	–	–	–	–
XGBoost	–	–	–	–	–
Isolation Forest	–	–	–	–	–

Key takeaways (*to be completed with your findings*):

- Which model gives the best Recall/Precision trade-off?
- How did SMOTE / class weights affect performance?
- How does the unsupervised Isolation Forest compare to supervised models?
- What is the cost-optimal decision threshold?

Future Improvements

- ☐ SHAP / LIME explainability for individual predictions
- ☐ Autoencoder / One-Class SVM / Local Outlier Factor comparison
- ☐ Cost-sensitive learning based on transaction amount
- ☐ Real-time streaming with Kafka
- ☐ Model monitoring and drift detection
- ☐ Docker & docker-compose deployment
- ☐ CI/CD with GitHub Actions
- ☐ User authentication for the dashboard
- ☐ Cloud deployment (AWS / GCP / Azure / Render)